

Type-Lowering: Text as a Roaring Bitmap, Proximity as a Comonad

Abstract

Montague’s *type-raising* sends an object upward in the type hierarchy, $x \mapsto \lambda f. f(x)$, turning data into a functional that consumes its former operators. This note develops the dual move, which we call *type-lowering*: rather than raising the operator to act on the text as it is, we lower the text down to the representational level of its building blocks. A token, which lived *inside* a sequence, becomes an object of the same kind as the building block — a set of positions, i.e. a bitmap. We show that (i) proximity search factors through this lowering and the factorization commutes; (ii) the round trip “lower-then-realize” is a closure operator, and hence an idempotent monad on the ordered domain of matches; and (iii) the proximity operation itself is the co-Kleisli arrow of a **Store** comonad, the spatial structure familiar from stencil computations and cellular automata. The physical processor never changes. What changes is the interpretation of the bitmap: from a code for an *object* to a code for a *sequence-as-set*, which relocates sequentiality from control flow into data layout, where the processor’s pre-existing word-level parallelism makes it free.

1 Two domains

Definition 1 (The sequence domain **Seq**). Objects are texts presented as ordered sequences of tokens, $t = \langle t_0, t_1, \dots, t_{n-1} \rangle$ with $t_i \in \Sigma$ for a token alphabet Σ . Operations are *algorithmic*: a proximity test is a control flow,

for $i \in \text{pos}(A)$: **if** $\text{pos}(B) \cap [i - k, i + k] \neq \emptyset$ **then emit** i .

Time is explicit: iteration order *is* the sequence structure; step $i + 1$ waits on step i .

Definition 2 (The bitmap domain **Bit**). Fix a position universe $\mathcal{U} = \{0, 1, \dots, n - 1\}$. Objects are characteristic vectors $\chi: \mathcal{U} \rightarrow \{0, 1\}$, equivalently subsets of $\mathcal{U}$, realized as (roaring) bitmaps. Operations are *extensional and position-free*: bitwise $\wedge$ (AND), $\vee$ (OR), shift_δ , and popcount . There is no iteration — only algebra on whole sets, with the machine word processing 64 coordinates per instruction.

Remark 3 (**Bit** is built on a monoid). $(\mathcal{P}(\mathcal{U}), \wedge, \mathcal{U})$ is a commutative monoid: $\wedge$ is associative and the full bitmap $\mathcal{U}$ (all ones) is the unit. This monoid is the arithmetic that everything below ultimately computes *with*; the monad and comonad organize *when* and *where* it is applied.

2 Lowering and realization

Definition 4 (Lowering $L: \mathbf{Seq} \rightarrow \mathbf{Bit}$). For a text t and a token $A \in \Sigma$,

$$L(t)(A) = \{p \in \mathcal{U} : t_p = A\}, \quad \text{i.e.} \quad \chi_A(p) = \mathbf{1}_{[t_p=A]}.$$

Each token collapses from “a symbol at a place in a sequence” to *its own indicator function over the position universe*. The text as a whole becomes a bundle of such bitmaps, one per token type. This is the lowering: the token, formerly an element *inside* the text, is now an object of the same kind as the building block — a set of positions.

Definition 5 (Realization $R: \mathbf{Bit} \rightarrow \mathbf{Seq}$).

$$R(\chi) = \langle p : \chi(p) = 1 \rangle \text{ in ascending order.}$$

This is the popcount-style walk that reads out the set members in order. It is the unique operation in $\mathbf{Bit}$ that *reintroduces time*: it serializes a spatial set into a sequence.

3 Proximity factors through the lowering

Definition 6 (Proximity, both ways). In $\mathbf{Seq}$, $\text{near}_k(A, B)$ is the control-flow predicate above. In $\mathbf{Bit}$, define

$$\text{near}_k^{\mathbf{Bit}}(A, B) = \left(\bigvee_{\delta=-k}^k \text{shift}_\delta(L(t)(A)) \right) \wedge L(t)(B).$$

The shifted union masks a window of radius k around every occurrence of A ; the final $\wedge$ keeps exactly those occurrences of B that fall inside some window.

Proposition 7 (Commuting square). *The following commutes:*

$$\begin{array}{ccc} \mathbf{Seq} & \xrightarrow{\text{near}_k} & \mathbf{Seq} \\ L \downarrow & & \uparrow R \\ \mathbf{Bit} & \xrightarrow{\text{shift}_k + \wedge} & \mathbf{Bit} \end{array}$$

that is,

$$\text{near}_k(A, B) = R\left(\text{near}_k^{\mathbf{Bit}}(A, B)\right).$$

Commutativity is precisely the correctness statement for the lowering: the sequential and the bitmap computations return identical matches.

4 Lower-then-realize is a monad

Write $C = R \circ L: \mathbf{Seq} \rightarrow \mathbf{Seq}$ for “lower the text, do the bitmap algebra, read back out.” We claim C is a closure operator, hence an idempotent monad on the order of matches.

Definition 8 (Monad, briefly). A monad on a category is an endofunctor T with natural transformations $\eta: \text{id} \Rightarrow T$ (unit) and $\mu: T^2 \Rightarrow T$ (multiplication) satisfying the associativity and unit laws. These are the two monoid laws lifted one level: a monad is a monoid in the category of endofunctors, with composition playing the role of the monoid product. On a poset, a monad is exactly a closure operator.

Proposition 9 (C is a closure operator). $C = R \circ L$ satisfies:

- (i) **Extensivity** $x \sqsubseteq C(x)$ — this is the unit η . Lowering-and-realizing loses no match; the positions survive the trip down to $\mathbf{Bit}$ and back up.

- (ii) **Idempotence** $C(C(x)) = C(x)$ — this is the multiplication μ in its strongest form. Equivalently $L \circ R \circ L = L$: two layers of bitmap coding flatten to one, and the bitmap representation is a fixed point.
- (iii) **Monotonicity** (functoriality) — more matches in yield more matches out; the operation respects the order.

Remark 10. The poetic “raising” of the CPU — imagining a single processor as a server farm — obeys none of these laws; it is an image. The lowering *has* the laws, and that is what makes it mathematics rather than metaphor.

5 Proximity is a Store comonad

Proximity is not a production operation; it is *observation in context*: stand at position p , look at the neighborhood $[p - k, p + k]$. That signature is comonadic.

Definition 11 (Comonad, briefly). A comonad is a monad with both arrows reversed: an endofunctor T with $\varepsilon: T \Rightarrow \text{id}$ (counit, “extract”) and $\delta: T \Rightarrow T^2$ (comultiplication, “duplicate/extend context”). Rule of thumb: monads *build up and flatten* (production); comonads *break down and focus* (observation in context).

Definition 12 (The **Store** comonad over $\mathcal{U}$). A **Store** object is a pair (f, p) of a content map $f: \mathcal{U} \rightarrow \{0, 1\}$ (our bitmap) and a focus $p \in \mathcal{U}$. Then

$$\begin{aligned}\varepsilon(f, p) &= f(p) && \text{(read the bit at the focus point),} \\ \delta(f, p) &= (q \mapsto (f, q), p) && \text{(attach to every point its whole context).}\end{aligned}$$

Proposition 13 (near_k is the co-Kleisli arrow of a stencil). *Let the local rule be*

$$r(f, p) = \bigvee_{\delta=-k}^k f(p + \delta), \quad r: \text{Store}(\mathcal{U}) \rightarrow \{0, 1\},$$

“is there an occurrence within radius k of the focus?” Its co-Kleisli extension $r^\square = r \circ \delta$ lifts the local rule to all positions simultaneously, and

$$\text{near}_k^{\text{Bit}}(A, B) = (r^\square \circ L(t)(A)) \wedge L(t)(B).$$

Concretely, $r^\square$ is exactly $\bigvee_{\delta=-k}^k \text{shift}_\delta$ on the bitmap: the δ of the comonad is the operator that turns “go to the next position, check the neighbor” (sequential control flow) into “every point already carries its neighborhood” (spatial, parallel observation).

6 What actually changed

Nothing about the processor changed. Its word-level / SIMD parallelism was always physically present. The lowering L relocates the *next*-relation: in **Seq** it lives in control flow (an order the machine must respect); in **Bit** it lives along the bit-position axis (a coordinate the machine touches in bulk). Set intersection illustrates the point. In **Seq** it is a morphism realized as control flow,

$$A \cap B = \text{for } i \in A: \text{ if } i \in B \text{ then emit } i,$$

a chain of dependent steps. In **Bit** the *same function* is a morphism realized as algebra, $\chi_A \wedge \chi_B$, sixty-four coordinates per instruction. The dependence between steps does not vanish because the machine got faster; it vanishes because L moved sequentiality out of the operator and into the geometry of the data, where it is free.

	builds up	breaks down
operation	monoid ($\wedge$, unit = $\mathcal{U}$)	—
functor	monad $R \circ L$ (closure: nothing lost, idempotent)	comonad Store (proximity: focus + neighborhood)
direction	lower the text, stabilize the representation	read each position in its context, in parallel

Type-raising the CPU is a dream the processor has about itself. Type-lowering the text is an actual change in what the bitmap *means*: a bitmap coding the number 5 and a bitmap coding the set $\{2, 4, 5\}$ can be the same bit string — the difference is only which functor, which reading of $\mathcal{U}$, we hold in mind. The lowering is the monad that stabilizes the representation; proximity is the comonad that spreads neighborhood across the universe; and the monoid underneath is what they both finally compute with. Same machine, new semantics.